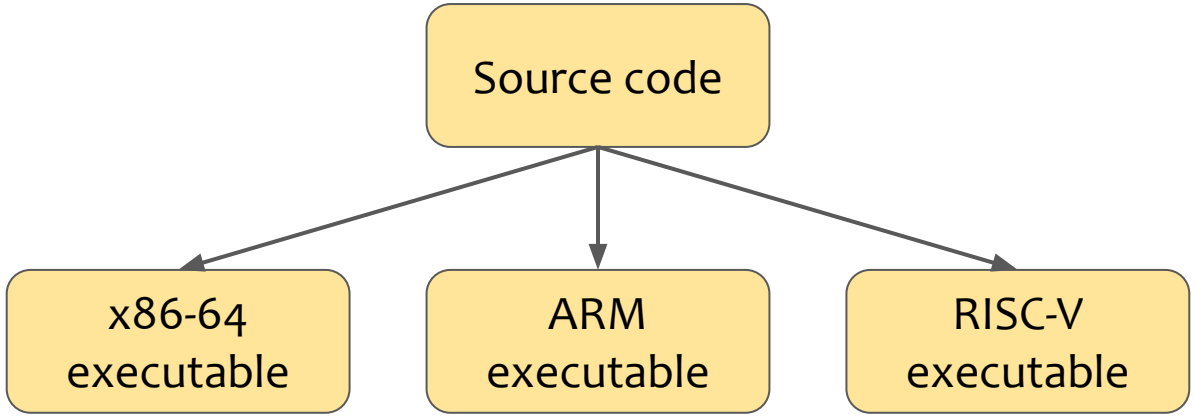
Specification-Driven Binary Translator

Robert Barinov
Advisors: Anders Choi, Martin Fink
Chair of Computer Systems
https://dse.in.tum.de/

24.03.2026

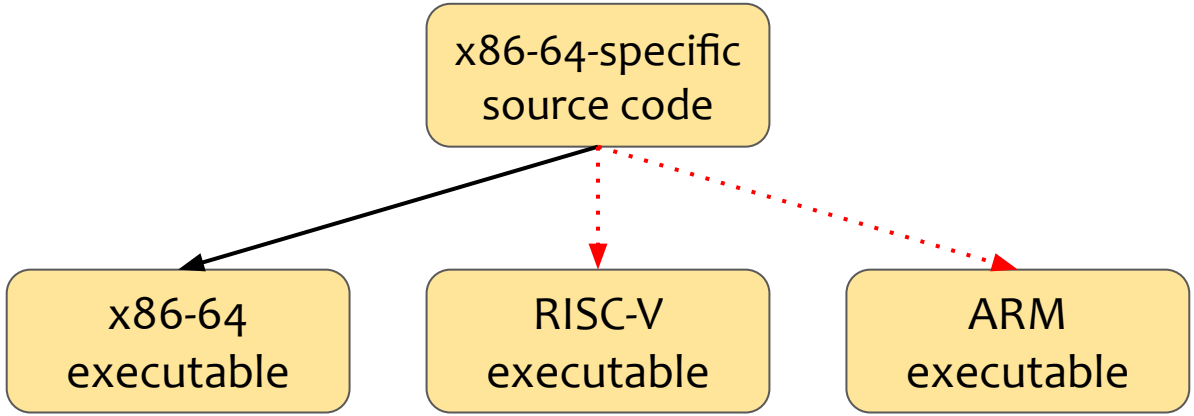
Binary Translation - Motivation

- Modern computing spans a wide range of architectures
  - x86-64, ARM, RISC-V…
- Software portability has become more important



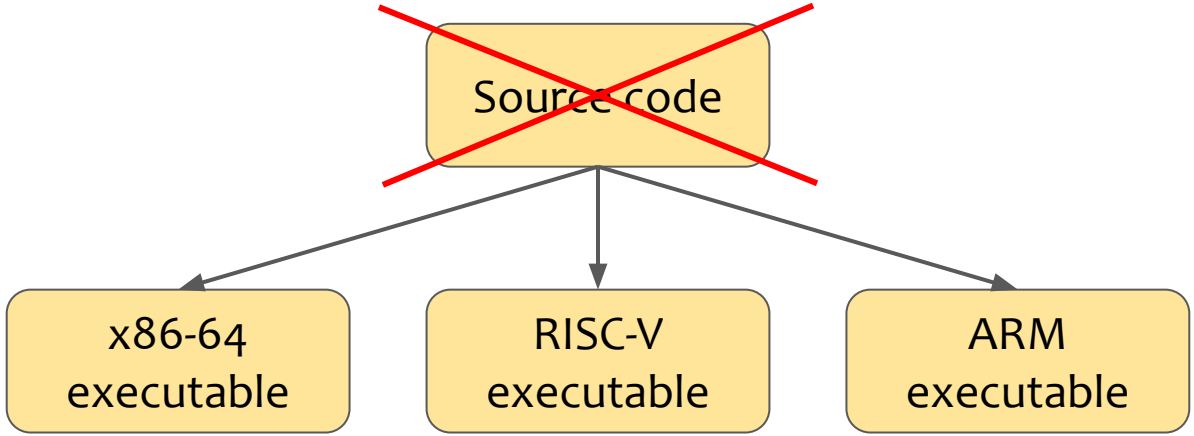
Binary Translation - Motivation

- Recompilation from source code is not always an option, since the source code may be
  - Architecture-specific (x86-64 SSE intrinsics)



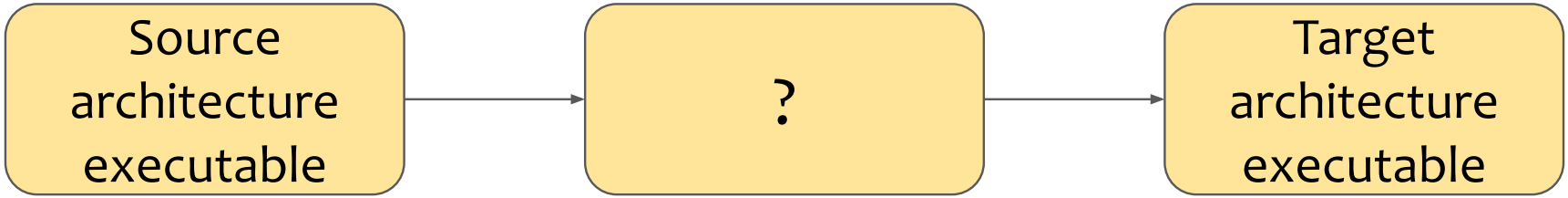
Binary Translation - Motivation

- Recompilation from source code is not always an option, since the source code may be
  - Unavailable (lost)



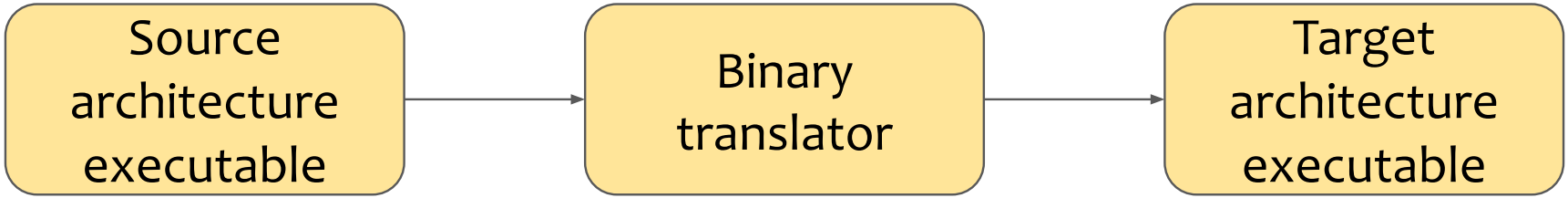
Binary Translation - Motivation

- **Compatibility is still desired!**



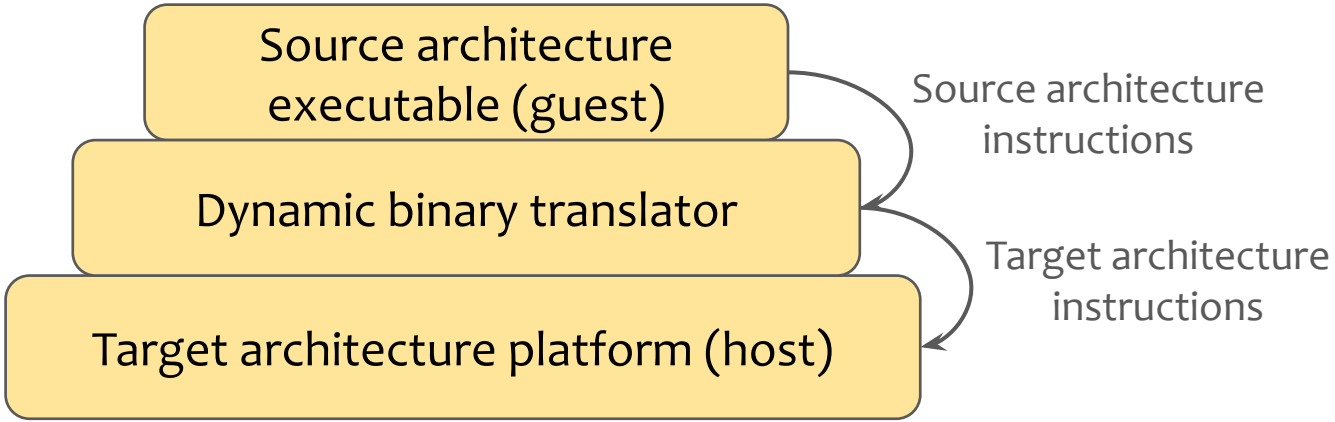
Binary Translation - Motivation

- Possible solution: **binary translation**
- Translation of instructions compiled for one ISA into instructions of another ISA without referring to the source code

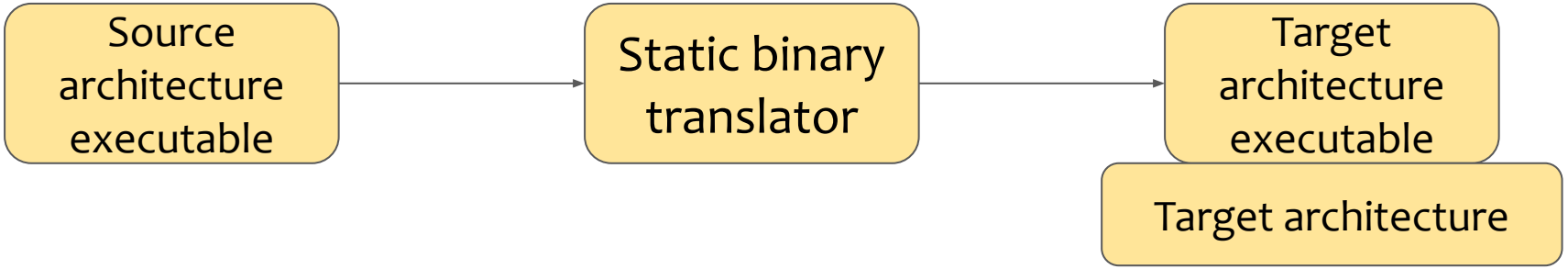


Dynamic Binary Translation (DBT)

- Translates instructions to the target architecture at execution time
- Example: QEMU user mode



Static Binary Translation (SBT)



- Translates an executable to the desired architecture ahead-of-time
- **Focus of this thesis**

Binary Lifting

- A core component of a static binary translator is **the binary lifter**
  - Lifts an executable's instructions to compiler IR sequences

Specification-Driven Tools - Motivation

- Problem: writing binary lifters manually
  - Scales poorly
    - Architecture extensions must be supported manually
  - Requires significant implementation effort
    - e.g. ARM specification has 14k pages [1]
  - Is error-prone
    - e.g. 22% of QEMU user mode bugs are mistranslation bugs [2]

[1] Arm Architecture Reference Manual
[2] Focaccia: Semantic Correctness Validation for CPU Emulators, Augoustis et al.

Specification-Driven Tools - Motivation

- Solution: **Specification-driven binary lifting**
  - Idea: generation of a binary lifter from an architecture specification
  - Prerequisite: requires a formal, machine-readable description of the target architecture

Machine Readable Architecture Specification

- ARM machine-readable specification (MRAS) is available since 2017 [3]
- Is a formal description of ARM in the Architecture Specification Language (ASL), describing
  - Encoding and decoding procedures
  - Instruction semantics

[3] Reid, Alastair. "Trustworthy Specifications of ARM® V8-A and v8-M System Level Architecture." 2016 Formal Methods in Computer-Aided Design (FMCAD), October 2016, 161–68. https://doi.org/10.1109/FMCAD.2016.7886675.

Airlift

- **Airlift** [4] - a specification-driven binary lifter by Anders Choi
  - Core parts generated from MRAS
- Foundation of this thesis

[4] Airlift: Choi, Anders. "Airlift: Binary Lifter Based on a Machine-Readable Architecture Specification." Master's Thesis, Technical University of Munich, 2025.

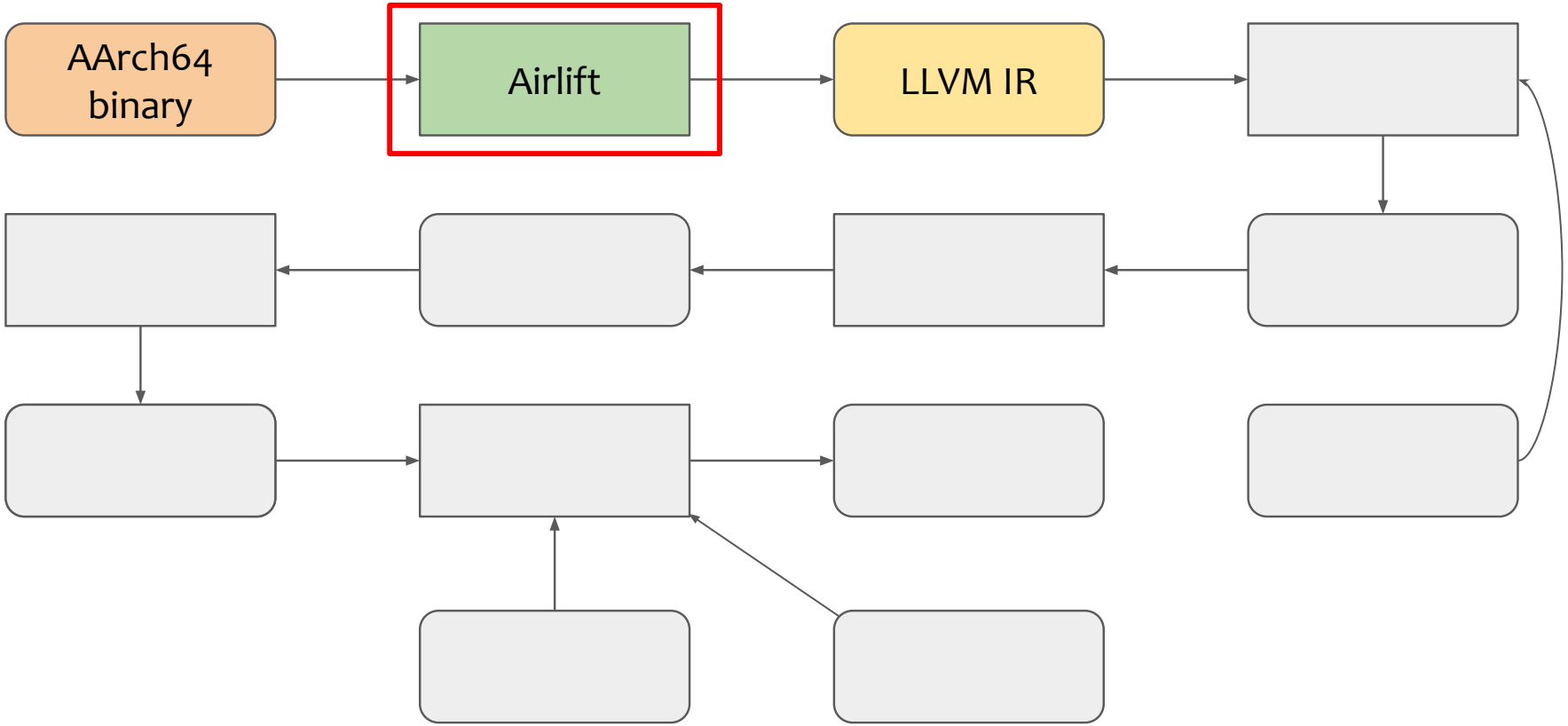
Project Goal

- A static binary translator has more components than just a lifter
- The goal of the thesis was to **extend** and **complement** **Airlift**, such that it can function as a part of a binary translator from AArch64 to x86-64
- Project name: **SGTranslator**

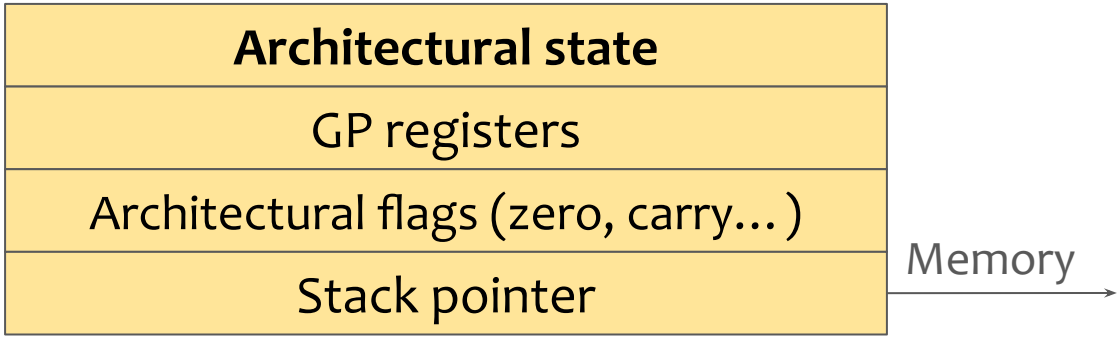
Outline

- ~~Motivation and background~~
- Design
- Evaluation
- Conclusion

Project Overview



Airlift - General Principle

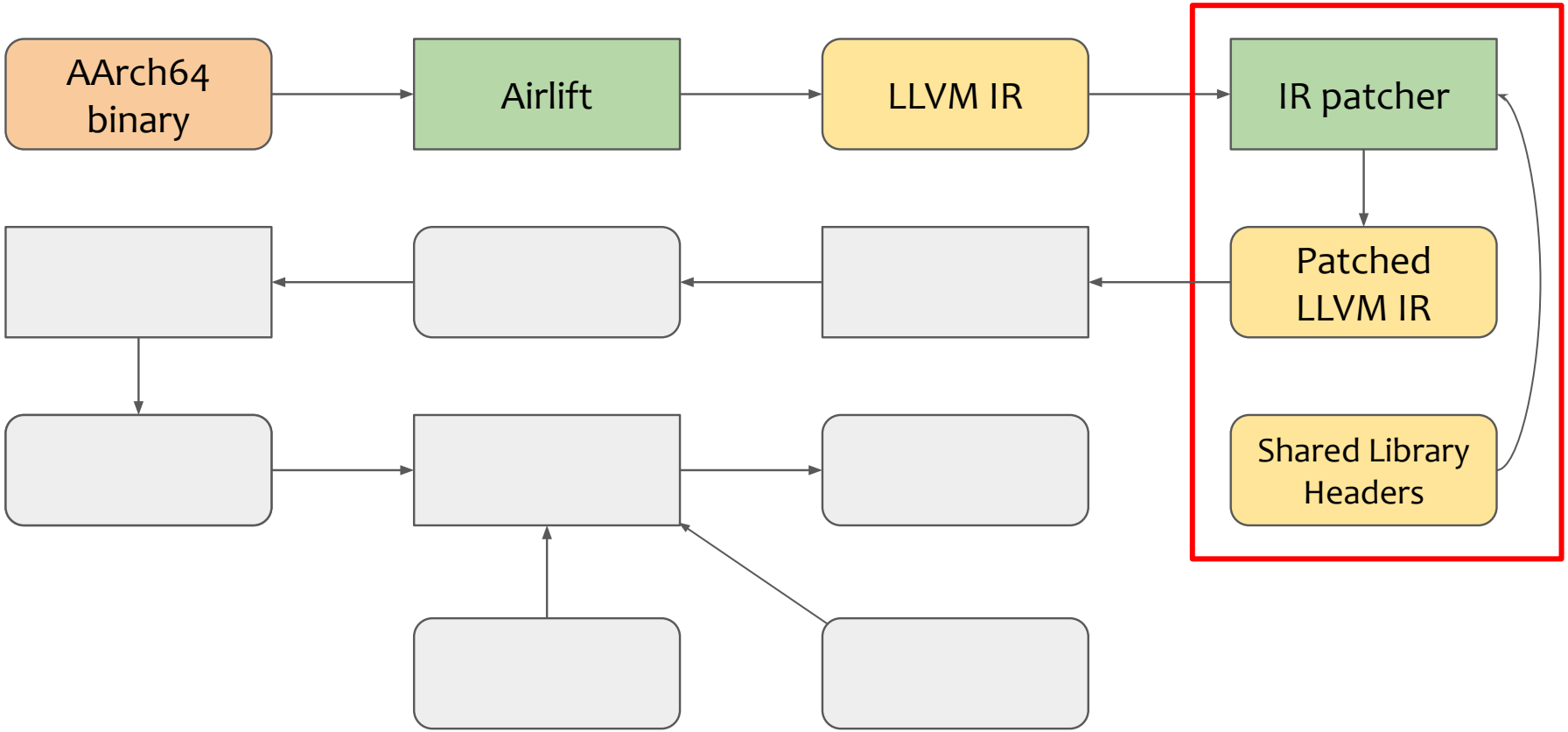


- Lifts AArch64 instructions to emulation-style LLVM IR
  - Airlift keeps a **state** structure, which emulates the processor state
  - The **state** contains fields for GP register values, FP register values, architectural flag values, the stack pointer...
  - Every machine instruction is lifted into a sequence which emulates its effects on hardware

Our Modifications to Airlift

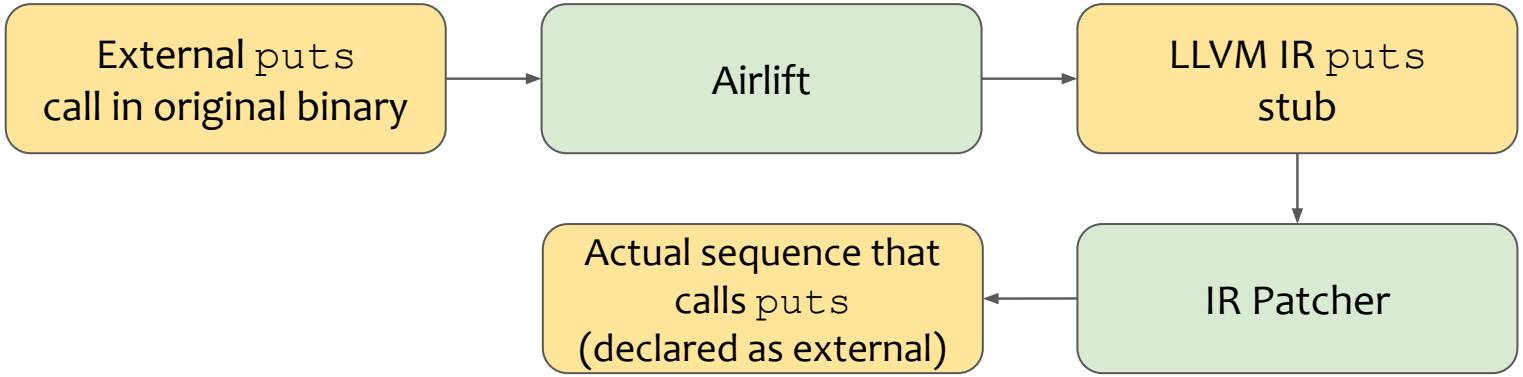
- Was modified to work together with other components of **SGTranslator**
  - Renames the old entry point, such that a new one can be inserted
  - Embeds a copy of the original AArch64 binary into the resulting LLVM IR
    - Used by instructions that read/write data encoded into the binary (e.g string literals)
  - Recognizes **shared library calls** and **system calls**, lifts them into dummies that will be handled later

Project Overview

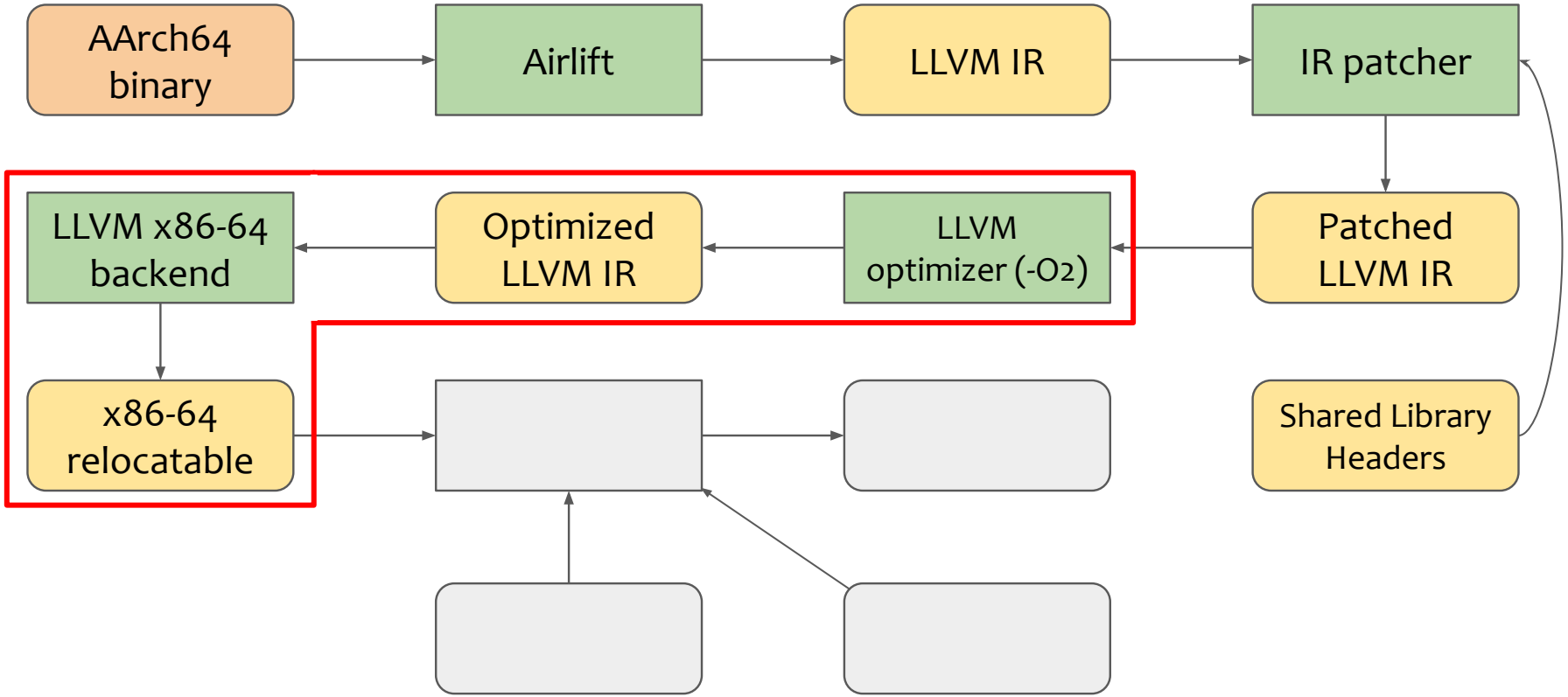


IR Patcher

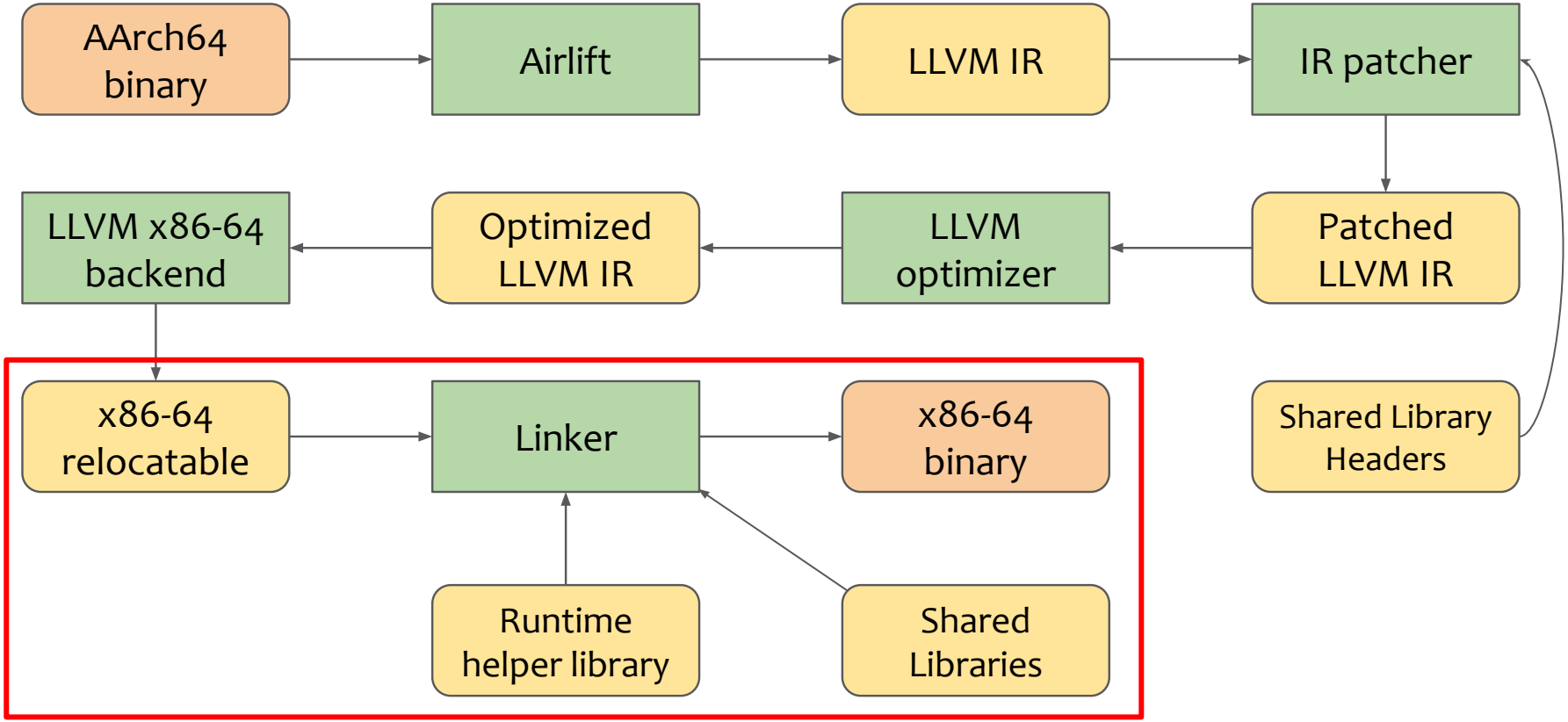
- Inserts a new entry point into the LLVM IR module, which will be used to initialize the guest binary state correctly
- Resolves calls to functions from external libraries to actual calls by parsing their headers, provided by the user



Project Overview



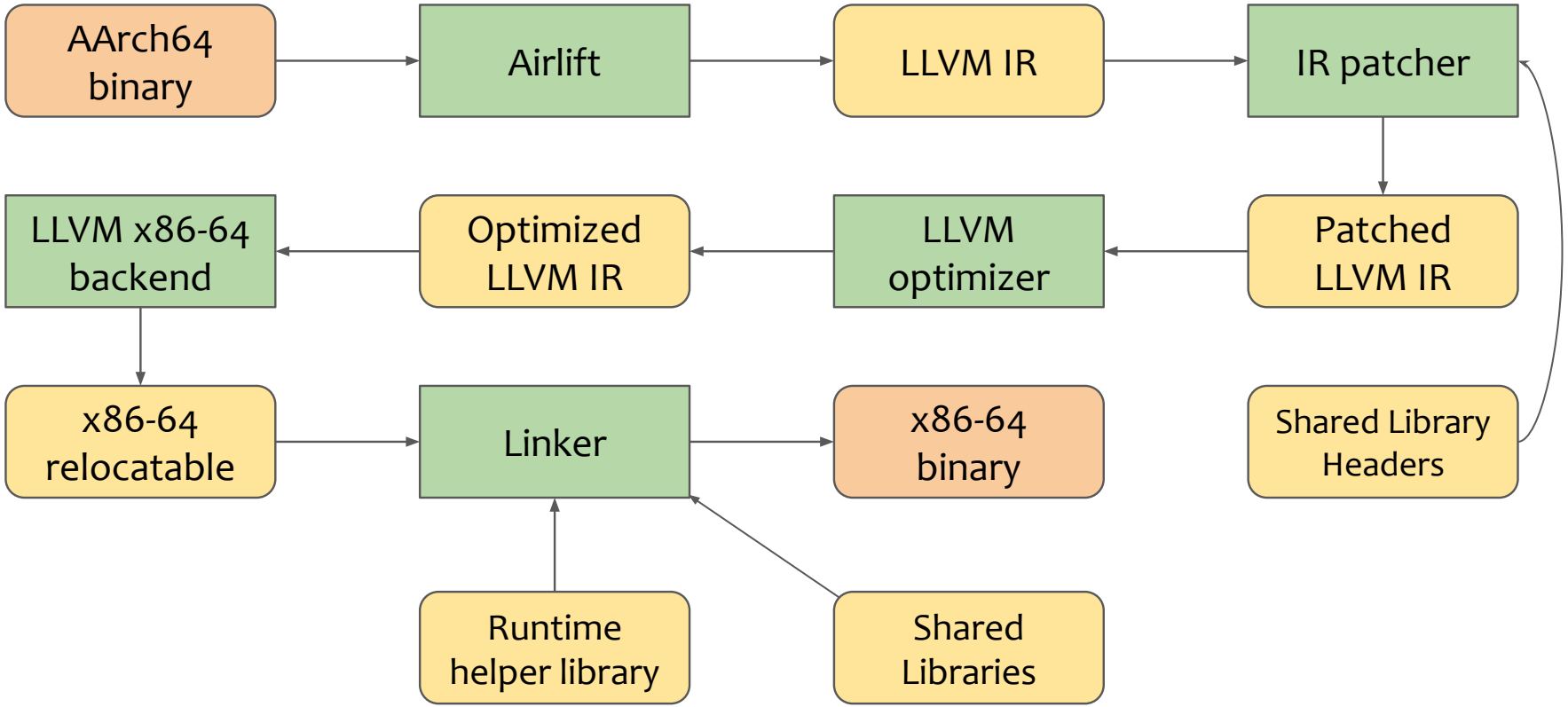
Project Overview



Runtime Helper Library

- Initializes the guest executable's state
  - Allocates a stack
  - Sets the emulated stack pointer correctly in **state**
- Calls the original entry point that was renamed by **Airlift**
- Provides a syscall handler function, which translates the dummy calls inserted by **Airlift** to actual x86-64 syscalls

Project Overview



Outline

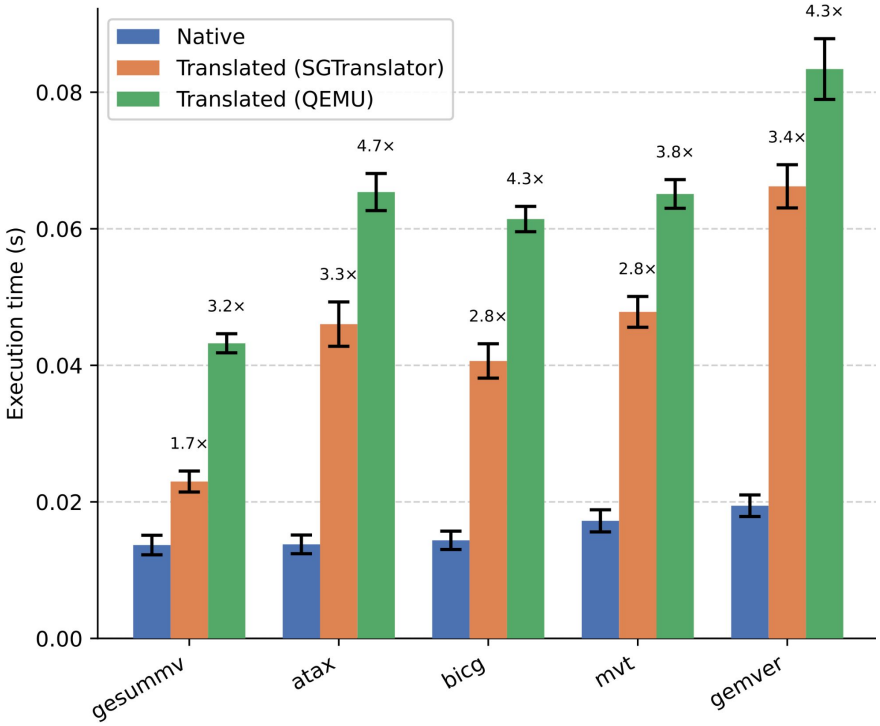
- ~~Motivation and background~~
- ~~Design~~
- Evaluation
- Conclusion

Performance Evaluation

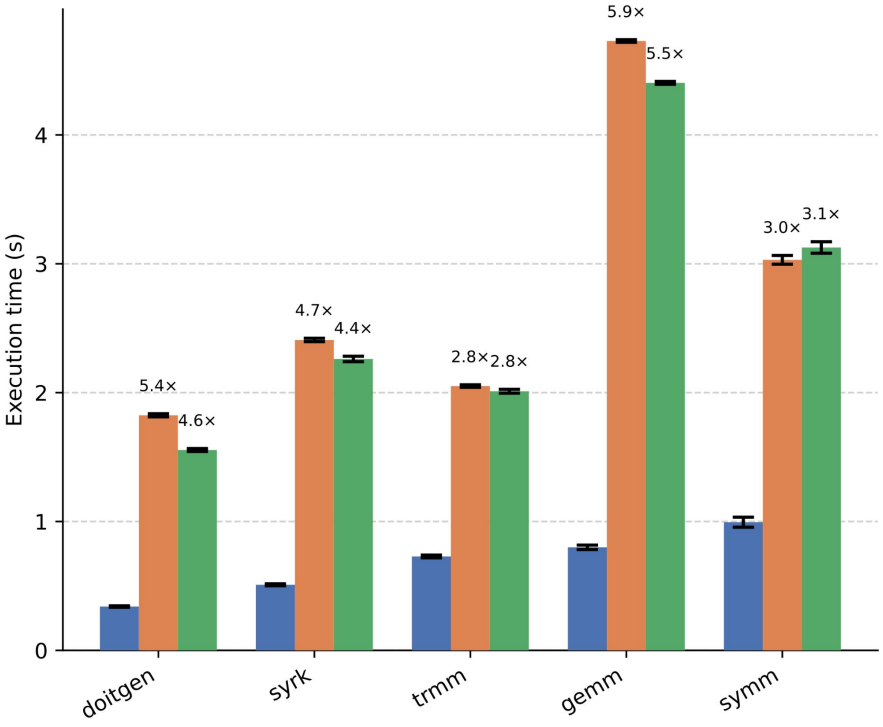
- How does **SGTranslator** perform compared to binaries compiled for x86-64 and to QEMU?
- **PolybenchC** [5]
  - 15/30 benchmarks that do not rely on FP computations
  - Integer-type only runs
- Experimental methodology:
  - Setup: AMD EPYC 7713P (64 cores, 3.72 GHz), 2 MiB L1, 32 MiB L2, 256 MiB L3, 1 TiB RAM (negligible memory pressure)
  - Single-core execution to avoid interference, 30 runs for every benchmark

[5] PolybenchC: https://www.cs.colostate.edu/~pouchet/software/polybench/

Performance Evaluation



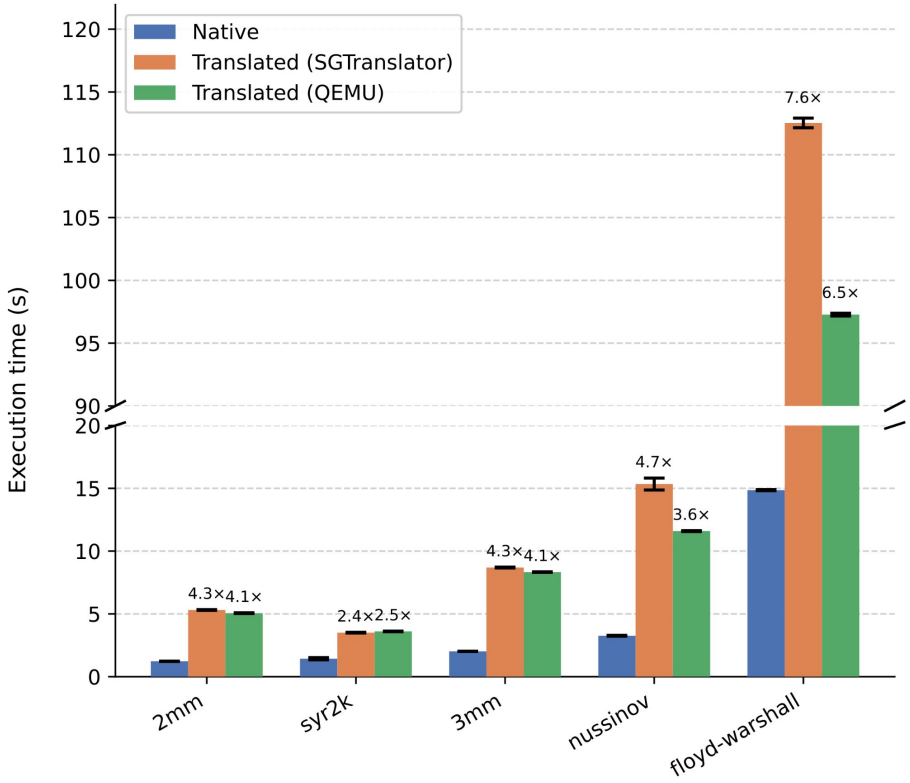
Fast benchmark results



Medium benchmark results

Performance Evaluation

- Geometric mean of overheads compared to native binaries: 3.686
- Realistically, we perform worse than QEMU
  - On average, we perform better than QEMU due to small benchmarks, where QEMU init time dominates
- Expected, since the IR **Airlift** produces is not performance-oriented
  - ASL function inlining everywhere → instruction cache overload



Slow benchmark results

Correctness Evaluation

- Methodology: comparison between translated binaries and QEMU
- Comparison of side effects (stdout, stderr, return code)
- Hand-written integration test suite, consisting of
  - Basic assembly tests
  - C programs as larger, more general tests
- **PolybenchC** was also checked for correctness
- All tests from both **PolybenchC** and our test suite pass
  - General sanity check of **SGTranslator**

Outline

- ~~Motivation and background~~
- ~~Design~~
- ~~Evaluation~~
- Conclusion

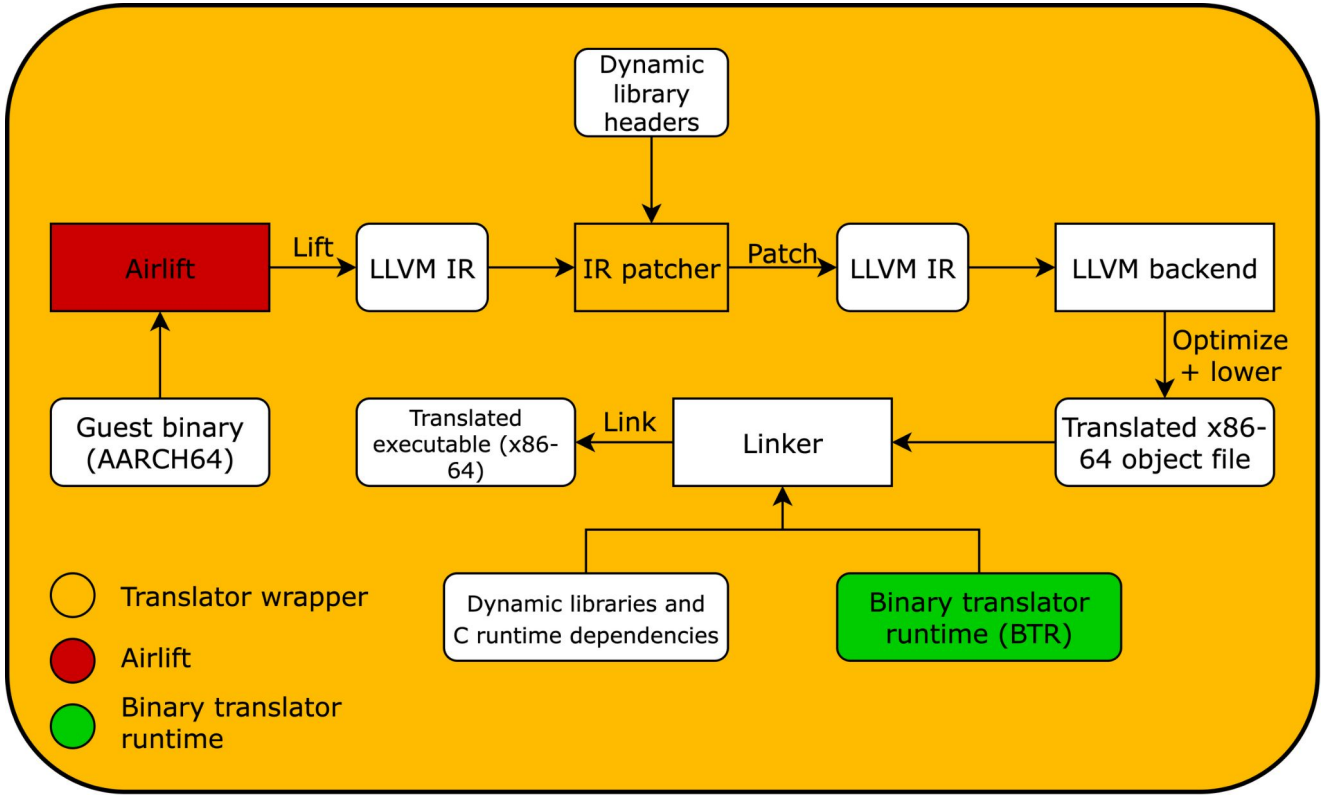
Future Work

- Support features left out due to time constraints
  - Globally visible symbols with external linkage
  - Calls to variadic external functions
  - FP computations
  - Translation of binaries generated from programming languages other than C and plain assembly
- Hybrid solution: integrate a **DBT** into **SGTranslator** to overcome key limitations of static translation
  - Dynamic jumps

Conclusion

- **SGTranslator** - the first static, specification-driven binary translator, driven by **Airlift**
  - Serves as a successful proof of concept
- Opens the possibility of future hybrid solutions to expand the set of programs we can translate

Backup slides


Dynamic library headers
Airlift
Lift
LLVM IR
IR patcher
Patch
LLVM IR
LLVM backend
Optimize + lower
Guest binary (AARCH64)
Translated executable (x86-64)
Link
Linker
Translated x86-64 object file
Dynamic libraries and C runtime dependencies
Binary translator runtime (BTR)
Translator wrapper
Airlift
Binary translator runtime